

In this project, we had to modify the xv6 operating system to implement a lottery scheduler. The idea of the lottery scheduler is that each process owns a number of tickets and each scheduling done allows the kernel to randomly select a winning ticket. So a process with more tickets will have a higher probability of being chosen. In this project we were required to modify the xv6 kernel, add two new system calls, update the scheduler, and write a test program to measure the CPU time distribution. And also create a graph to analyze how often each process ran in relative to its ticket count.

Proc.h = In this file, I added int tickets for the number of lottery tickets the process owns, and int ticks for the number of times the scheduler selected a process. I did this because tickets will allow for the scheduler to track CPU shares and the ticks will track the actual CPU usage.

```
struct file *ofile[NOFILE]; // Open files
struct inode *cwd;          // Current directory
char name[16];              // Process name (debugging)
int ticks;                  // times schedules
int tickets;                // lottery tickets
};
```

Proc.c = In proc.c, i replaced the default round robin scheduler with a lottery scheduler, in the scheduler() function, i added logic to sum the total number of ticks across all runnable processes, generate a random winning ticket, and select the process whose cumulative ticket range contains the winning number. I also incremented the selected process's ticks field each time it was chosen. This transforms the scheduler from round robin to lottery scheduling. Also in allocproc(), I initialized each new process with ticket = 1 and ticks = 0. In fork(), I ensure that child processes are able to inherit their parents' ticket count.

```

static struct proc*
allocproc(void)
{
    struct proc *p;
    char *sp;

    acquire(&ptable.lock);

    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
        if(p->state == UNUSED)
            goto found;

    release(&ptable.lock);
    return 0;

found:
    p->state = EMBRYO;
    p->pid = nextpid++;
    p->tickets = 1;
    p->ticks = 0;

    release(&ptable.lock);

    // Allocate kernel stack.
    if((p->kstack = kalloc()) == 0){
        p->state = UNUSED;
        return 0;
    }
    sp = p->kstack + KSTACKSIZE;

    // Leave room for trap frame.

```

```

void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();
    c->proc = 0;

    for(;;){
        // Enable interrupts on this processor.
        sti();

        // Loop over process table looking for process to run.
        acquire(&ptable.lock);
        int totaltickets = 0;
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
            if(p->state == RUNNABLE)
                totaltickets += p->tickets;
        }

        // Switch to chosen process. It is the process's job
        // to release ptable.lock and then reacquire it
        // before jumping back to us.

        if(totaltickets == 0){
            release(&ptable.lock);
            continue;
        }

        int winner = rand() % totaltickets + 1;
        int count = 0;
        struct proc *chosen = 0;

        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
            if(p->state != RUNNABLE)
                continue;
            count += p->tickets;

```

```

count += p->tickets;
if(count >= winner){
    chosen = p;
    break;
}
}

if(chosen){
    p = chosen;
    c->proc = p;
    switchvm(p);
    p->state = RUNNING;
    p->ticks++;

    swtch(&(c->scheduler), p->context);
    switchkvm();

    // Process is done running for now.
    // It should have changed its p->state before coming back.
    c->proc = 0;
}

release(&ptable.lock);
}
}

```

Sysproc.c = Added 2 new system calls. the sys-settickets(), validated input and updated the calling process's ticket count. sys_getpinfo() validates the user pointer and fills a pstat struct with pid, tickets, ticks, and inuse flags. They act as the bridge between user request and kernel scheduling date.

```

int
sys_settickets(void){
    int n;
    if(argint(0, &n) < 0 || n < 1)
        return -1;

    myproc()->tickets = n;
    return 0;
}

```

```

int
sys_getpinfo(void){
    struct pstat *ps;

    if(argptr(0, (void*)&ps, sizeof(*ps)) < 0)
        return -1;
    acquire(&ptable.lock);
    struct proc *p;
    int i = 0;
    for (p = ptable.proc; p < &ptable.proc[NPROC] && i < NPROC; p++, i++){
        if (p->state == UNUSED){
            ps->inuse[i] = 0;
            ps->pid[i] = 0;
            ps->tickets[i] = 0;
            ps->ticks[i] = 0;
        } else {
            ps->inuse[i] = 1;
            ps->pid[i] = p->pid;
            ps->tickets[i] = p->tickets;
            ps->ticks[i] = p->ticks;
        }
    }
    release(&ptable.lock);
    return 0;
}

```

Syscall.c = Added entries to the system call table for settickets and getpinfo. It allows the xv6 to route the syscall numbers to the correct kernel functions when the user invokes it.

```

extern int sys_settickets(void);
extern int sys_getpinfo(void);

```

```

[SYS_settickets] sys_settickets,
[SYS_getpinfo] sys_getpinfo,
};

```

Syscall.h = Added syscall numbers for SYS_settickets and SYS_getpinfo. They are identifiers that allow the kernel and user to agree on how the new system calls will be invoked.

```

#define SYS_settickets 26
#define SYS_getpinfo 27

```

User.h = Added function prototypes that makes the system calls available to user programs and allows them to request ticket changes and retrieve scheduling data.

```
int settickets(int);  
int getpinfo(struct pstat *);
```

usys.S = added user system call stubs that generate the correct trap instructions so user programs can invoke the new system calls.

```
SYSCALL(settickets)  
SYSCALL(getpinfo)
```

Pstat.h = contains the required struct used by getpinfo to return process statistics and inspect scheduler behavior.

```
GNU nano 8.4  
#ifndef _PSTAT_H_  
#define _PSTAT_H_  
  
#include "param.h"  
  
struct pstat {  
    int inuse[NPROC];  
    int tickets[NPROC];  
    int pid[NPROC];  
    int ticks[NPROC];  
};  
  
#endif
```

Lotterytest.c = This is a test program to evaluate the lottery scheduler to see if it runs well. It forks 3 processes, assigns tickets 30,20,10, runs CPU bound loops to accumulate ticks, parents sleeps to allow child to run, calls `getinfo()` to retrieve scheduling data, then prints each child's pid, ticket and tick counts.

```
#include "types.h"
#include "stat.h"
#include "user.h"
#include "pstat.h"
void
spin(char *name)
{
    int i;
    printf(1, "\n%s starting\n", name);

    for(i = 0; i < 500000000; i++){
        if(i % 100000000 == 0)
            printf(1, "%s running\n", name);
    }
    printf(1, "%s done\n", name);
}

int
main(void)
{
    int pid1, pid2, pid3;

    pid1 = fork();
    if(pid1 < 0){
        printf(1, "fork for A failed\n");
        exit();
    }
    if(pid1 == 0){
        settickets(30);
        spin("A-30");
        exit();
    }

    pid2 = fork();
    if(pid2 < 0){
        printf(1, "fork for B failed\n");
        exit();
    }
    if(pid2 == 0){
        settickets(20);
        spin("B-20");
        exit();
    }

    pid3 = fork();
    if(pid3 < 0){
        printf(1, "fork for C failed\n");
        exit();
    }
    if(pid3 == 0){
        settickets(10);
        spin("C-10");
        exit();
    }

    printf(1, "\nparent: A=%d B=%d C=%d\n", pid1, pid2, pid3);
    sleep(2000);

    struct pstat ps;
    if(getpinfo(&ps) < 0){
        printf(1, "getpinfo failed\n");
        exit();
    }

    int i;
    for(i = 0; i < NPROC; i++){
        if(ps.inuse[i] &&
            (ps.pid[i] == pid1 || ps.pid[i] == pid2 || ps.pid[i] == pid3)){
```

```
        printf(1, "pid %d tickets %d ticks %d\n",
                ps.pid[i], ps.tickets[i], ps.ticks[i]);
    }
}
wait();
wait();
wait();

exit();
}
```

Qemu runs successfully

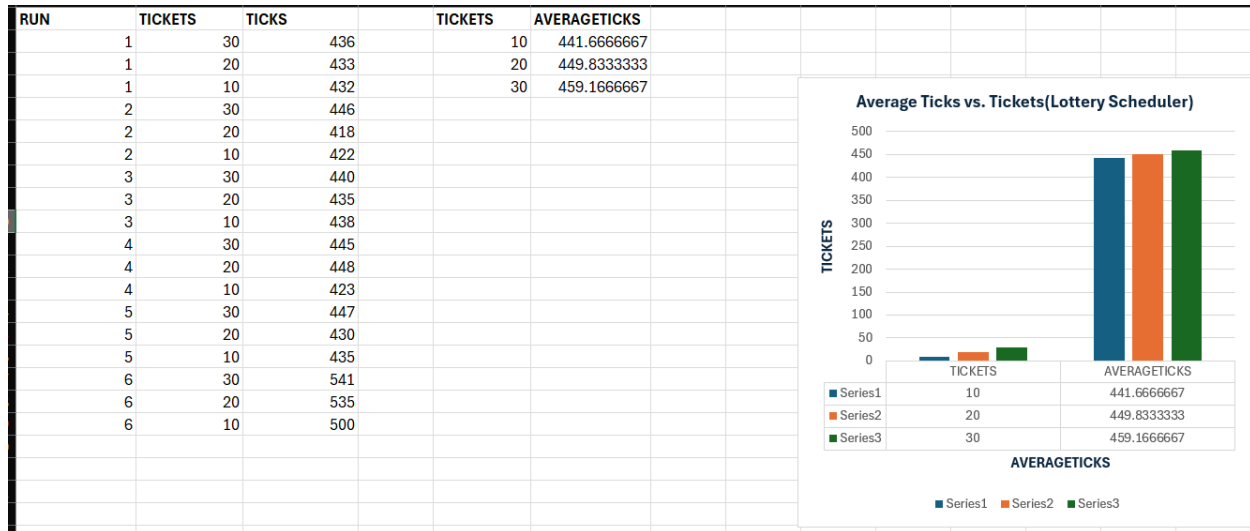
```

vm.x.o vm.x.c
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -fno-omit-frame-pointer -fno-stack-protector -fno-pie -no-p
qlock.o qlock.c
gcc -m32 -gdwarf-2 -Wa,-divide -c -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -fno-omit-frame-pointer -fno-stack-protector -fno-pie -no-p
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -fno-omit-frame-pointer -fno-stack-protector -fno-pie -no-p
-nostdinc -I. -c entryother.S
ld -m elf_i386 -N -e start -Ttext -o bootblock.o bootblock.o entryother.o
ld: warning: bootblockother.o has a
objcopy -S -O binary -j .text bootb
objdump -S bootblockother.o > entry
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 -fno-omit-frame-pointer -fno-stack-protector -fno-pie -no-p
-c -I. -c initcode.S
ld -m elf_i386 -N -e start -Ttext -o initcode.o initcode.o
ld: warning: initcode.out has a LOA
objcopy -S -O binary initcode.out icpu0: starting 0
objdump -S initcode.o > initcode.asm
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
ld -m elf_i386 -T kernel.ld -o kinit: starting sh
ipe.o proc.o sleeplock.o spinlock.o
ry initcode entryother
objdump -S kernel > kernel.asm
objdump -t kernel | sed '1,/SYMBOL
dd if=/dev/zero of=xv6.img count=10
10000+0 records in
10000+0 records out
5120000 bytes (5.1 MB, 4.9 MiB) cop
dd if=bootblock of=xv6.img conv=not
1+0 records in
1+0 records out
512 bytes copied, 0.000572627 s, 51
dd if=kernel of=xv6.img seek=1 conv=notrunc
421+1 records in
421+1 records out
215980 bytes (216 kB, 211 KiB) copied, 0.00761856 s, 31 MB/s
qemu-system-i386 -serial mon:stdio -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=disk,format=raw -smp
xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh

```

This is an out i got for by running lotterytest.c in qemu

```
objcopy -S -O binary initcode.out initcode
objdump -S initcode.o > initcode.asm
ld -m elf_i386 -T kernel.ld -o kernel entry.o bio.o console.o exec.o file.o fs.o ide.o ioapic.o kalloc.o kbd.o lapic.o log.o main.o mp.o picirq.o
ipe.o proc.o sleeplock.o spinlock.o syscalls.o vm.o vm.o vm.o vm.o vm.o vm.o vm.o vm.o vm.o vm.o vm.o vm.o vm.o vm.o vm.o vm.o vm.o vm.o vm.o vm.o vm.o
ry initcode entryother
objdump -S kernel > kernel.asm
objdump -t kernel | sed '1,/SYMBOL TABLE/ d'
dd if=/dev/zero of=xv6.img count=1000000
1000000 records in
1000000 records out
5120000 bytes (5.1 MB, 4.9 MiB) copied, 0.000456526 s, 512 bytes copied, 0.000456526 s, 512 bytes copied
dd if=bootblock of=xv6.img conv=not
1+0 records in
1+0 records out
512 bytes copied, 0.000456526 s, 512 bytes copied
dd if=kernel of=xv6.img seek=1 conv=not
421+1 records in
421+1 records out
215980 bytes (216 kB, 211 KiB) copied, 0.000456526 s, 512 bytes copied
qemu-system-i386 -serial mon:stdio -smp 2 -m 512M -hda xv6.img -cdrom raw -smp 2 -m 512M
xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
init: starting sh
$ lotterytest
parent: A=4 B=5 C=6
B-20 running
B-20 running
A-30 running
A-30 running
A-30B-20 done
done
C-10 running
C-10 running
C-10 done
pid 4 tickets 30 ticks 6
pid 5 tickets 20 ticks 5
pid 6 tickets 10 ticks 6
$
$
```

To be able to do the graph, I ran the lottery scheduler test 6 times and recorded the number of ticks each process received. I averaged the tick count for each ticket value (10,20,30) and plotted them in excel. The graph showed that the process with more tickets tends to have more ticks. Which is an expected behavior of a lottery scheduler. Processes with more tickets receive slightly more CPU time on average. Although in the graph the differences are small. Which might have happened because if a process wins more often, each win has an increment of exactly 1 which compresses the difference and also as a result of the long running CPU processes which makes schedulers become extremely fair over time resulting in a small but consistent upward trend as seen in the graph. But even still, the graph shows a consistent upward trend in tick counts as ticket value increases, which shows the nature of a lottery scheduler.